REVISION CHEAT SHEET:

WEB API / REST

ASP.NET Core Web APIs and REST

Interview-ready notes · examples · C# · diagrams · clickable index

INDEX / TABLE OF CONTENTS

Click a section name to jump to that section.

1. What Is a Web API?
2. HTTP Is the Foundation of a Web API
3. What Is REST?
4. Core REST Constraints
5. Client-Server Separation
6. Stateless Requests
7. Resources
8. Resource URI Design
9. Plural Resource Names
10. Collection vs Individual Resource
Diagram 1 — Resource-Oriented API
PART 1 — HTTP METHODS
11. GET
12. Do Not Change Business State with GET
13. POST
14. POST Is Not Automatically Idempotent
15. PUT
16. PUT Is Idempotent
17. PUT vs PATCH
18. PATCH and Domain Behavior
19. DELETE
20. DELETE Is Idempotent
21. Safe vs Idempotent
PART 2 — HTTP STATUS CODES
22. Why Status Codes Matter
23. 200 OK
24. 201 Created
25. 202 Accepted
26. 204 No Content
27. 400 Bad Request
28. 401 Unauthorized
29. 403 Forbidden
30. 404 Not Found
31. 405 Method Not Allowed
32. 409 Conflict
33. 412 Precondition Failed
34. 415 Unsupported Media Type
35. 422 Unprocessable Content
36. Status-Code Design Rule
PART 3 — CONTROLLER-BASED APIs
37. ControllerBase
38. Controller vs ControllerBase
39. ApiController Attribute
40. Automatic Validation Response
41. Example ValidationProblemDetails
42. Binding Source Inference
PART 4 — MODEL BINDING
43. What Is Model Binding?
44. FromRoute
45. FromQuery
46. FromBody
47. FromHeader
48. FromServices
49. Model Binding Conversion Failure
PART 5 — REQUEST VALIDATION
50. DataAnnotations
51. Validation Happens After Binding
52. Request Validation vs Domain Validation
53. DTO Validation Is Boundary Validation
54. Non-Nullable Reference Types and Validation
PART 6 — DTO DESIGN
55. Do Not Expose EF/Domain Entities Directly
56. Response DTO
57. Separate Request and Response DTOs
58. Update DTO Should Also Be Deliberate
PART 7 — CONTROLLER ACTION RETURN TYPES

59. Specific Type
60. IActionResult
61. ActionResult<T>
62. CreatedAtAction
63. Why CreatedAtAction Is Better Than Ok for Creation
64. IAsyncEnumerable<T>
PART 8 — MINIMAL APIs
65. Minimal API Example
66. Minimal API Return Values
67. Results
68. TypedResults
69. Results<T1, T2>
70. Controller vs Minimal API
PART 9 — CONTENT NEGOTIATION
71. What Is Content Negotiation?
72. Default Response Formats
73. Accept Header
74. Content-Type vs Accept
75. Consumes
76. Produces
PART 10 — JSON SERIALIZATION
77. System.Text.Json
78. Configure JSON
79. Enum Serialization
PART 11 — PROBLEM DETAILS
80. Standard Error Shape
81. ProblemDetails Example
82. Do Not Leak Internal Details
PART 12 — PAGINATION
83. Never Return an Unbounded Collection by Default
84. Offset Pagination
85. Paged Response
86. Limit Page Size
87. Cursor/Keyset Pagination
PART 13 — FILTERING, SORTING, SEARCH
88. Filtering
89. Sorting
90. Search
91. Query DTO
PART 14 — IDEMPOTENCY
92. Why Idempotency Matters
93. Idempotency Key Concept
94. Idempotency Is a Business Requirement Too
PART 15 — CONCURRENCY AND REQUESTS
95. Lost Update Problem
96. EF Core Concurrency + HTTP
97. 409 vs 412 for Concurrency
PART 16 — CACHING
98. GET Responses Can Be Cacheable
99. Don't Cache Sensitive Responses Carelessly
PART 17 — API VERSIONING
100. Why APIs Need Versioning Strategy
101. Versioning Approaches
102. Prefer Backward-Compatible Evolution
103. Database Version Is Not API Version
PART 18 — OPENAPI
104. What Is OpenAPI?
105. First-Party OpenAPI Support
106. Development-Only OpenAPI Endpoint
107. Response Metadata
108. OpenAPI Should Match Reality
PART 19 — CANCELLATION AND ASYNC API DESIGN
109. Request CancellationToken
110. Propagate Cancellation
PART 20 — PERFORMANCE
111. API Performance Starts With Response Shape
112. Read Projection
113. Avoid N+1 API Processing
114. Compression vs Huge Payloads
115. Streaming vs Pagination
PART 21 — SECURITY BOUNDARIES
116. DTO Validation Is Not Authorization
117. Never Trust Tenant IDs from Clients
118. Over-Posting
119. ASP.NET Core 10 Cookie Behavior for APIs
PART 22 — COMPLETE FUNDRAISING CONTROLLER
120. Request and Response Contracts
121. Controller
Diagram 2 — Create Gift Flow
122. Search Endpoint
PART 23 — MINIMAL API EQUIVALENT
123. Route Group
124. Get Gift
PART 24 — API TESTING
125. Integration Test the Contract

126. Test Creation Semantics
127. Validation Test
PART 25 — COMMON MISTAKES
128. Verbs Everywhere in URLs
129. Using GET to Change State
130. Returning 200 for Every Result
131. Returning Entity Framework Entities
132. One DTO for Everything
133. Domain Logic in Controller
134. Unbounded GET Endpoints
135. Accepting TenantId Blindly
136. Blind PUT/PATCH Mapping
137. Ignoring Idempotency in Financial Endpoints
138. Treating DataAnnotations as Domain Rules
139. Versioning Too Early
140. Versioning Too Late
141. OpenAPI Without Accurate Response Metadata
PART 26 — BEST PRACTICES
PART 27 — PRACTICAL EXERCISES
Exercise 1 — Design Gift Routes
Exercise 2 — Create Endpoint
Exercise 3 — Automatic Validation
Exercise 4 — Binding Sources
Exercise 5 — Pagination
Exercise 6 — Filtering
Exercise 7 — Sorting
Exercise 8 — ProblemDetails
Exercise 9 — Idempotency
Exercise 10 — Concurrency
Exercise 11 — Minimal API
Exercise 12 — OpenAPI
PART 28 — EXACTLY 25 INTERVIEW QUESTIONS
1. What is a REST API?
2. What is a resource in REST?
3. What is the difference between GET and POST?
4. What does idempotent mean in HTTP?
5. What is the difference between PUT and PATCH?
6. Why should GET not change server state?
7. When should an API return 201 Created?
8. What is the difference between 200 and 204?
9. What is the difference between 400, 404, and 409?
10. What is [ApiController] ?
11. What is model binding?
12. What is the difference between model binding and model validation?
13. What are [FromRoute] , [FromQuery] , and [FromBody] ?
14. Why should API DTOs usually be separate from domain/EF entities?
15. What is ActionResult<T> ?
16. When should you use IActionResult ?
17. What is the difference between Results and TypedResults in Minimal APIs?
18. What is content negotiation?
19. What is ProblemDetails?
20. How should pagination be designed?
21. What is API idempotency and why is it important for financial APIs?
22. How can ETags help with concurrent updates?
23. What is OpenAPI in ASP.NET Core?
24. How should API versioning be approached?
25. How would you design the Fundraising Platform Web API?
PART 29 — QUICK REVISION
PART 30 — FINAL END-TO-END FUNDRAISING API FLOW

1. What Is a Web API?

A Web API exposes application capabilities over HTTP.

For your Fundraising Platform, clients such as:

EXAMPLE / FLOW

```
Angular application

Mobile application

Partner integration

Internal service
```

```
Automation process
```

WEB API / REST

can communicate with the backend using requests such as:

EXAMPLE / FLOW

```
GET /api/gifts/125
```

or:

EXAMPLE / FLOW

```
POST /api/gifts
Content-Type: application/json
```

The ASP.NET Core application receives that HTTP request, maps it to an endpoint, executes application/domain logic, and returns an HTTP response.

Conceptually:

EXAMPLE / FLOW

```
Client
   ↓
HTTP Request
   ↓
ASP.NET Core API
   ↓
Controller / Minimal API
   ↓
Application Layer
   ↓
Domain
   ↓
Infrastructure
   ↓
HTTP Response
```

ASP.NET Core currently supports both **controller-based APIs** and **Minimal APIs**.

2. HTTP Is the Foundation of a Web API

A Web API is built around an HTTP request and HTTP response.

A request contains information such as:

EXAMPLE / FLOW

```
HTTP method

URI

Headers

Query string

Request body

Authentication information
```

Example:

EXAMPLE / FLOW

```
POST /api/gifts
Content-Type: application/json
Authorization: Bearer <token>

{
  "constituentId": 100,
  "campaignId": 20,
  "amount": 250.00
}
```

A response contains:

EXAMPLE / FLOW

```
Status code

Headers

Optional body
```

Example:

EXAMPLE / FLOW

```
HTTP/1.1 201 Created
Location: /api/gifts/501
Content-Type: application/json

{
  "giftId": 501,
  "amount": 250.00,
  "status": "Pending"
}
```

Understanding HTTP semantics is more important than memorizing controller attributes.

3. What Is REST?

REST means:

> **Representational State Transfer**

REST is an architectural style for networked systems.

It is not:

EXAMPLE / FLOW

```
a C# library

an ASP.NET Core feature

a JSON format

a database pattern

a synonym for "HTTP API"
```

An API can use HTTP and JSON without being particularly RESTful.

4. Core REST Constraints

The classic REST constraints include:

EXAMPLE / FLOW

```
Client-server separation

Stateless communication

Cacheability

Uniform interface

Layered system

Code-on-demand — optional
```

For normal backend APIs, the most practically important concepts are:

EXAMPLE / FLOW

```
Resource-oriented URLs

HTTP method semantics

Stateless requests

Standard HTTP status codes

Representations of resources

Consistent contracts
```

5. Client-Server Separation

The client and server have different responsibilities.

Client:

EXAMPLE / FLOW

```
Presentation
User interactions
Calling API
Displaying results
```

Server:

EXAMPLE / FLOW

```
Business rules
Authorization
Persistence
Integrations
Domain processing
```

For your application:

EXAMPLE / FLOW

```
Angular
   ↓
Fundraising API
```

```
    ↓
SQL Server
```

Angular should not directly know:

EXAMPLE / FLOW

```
EF Core entities

SQL schema details

Gift business invariants
```

The API boundary isolates those concerns.

6. Stateless Requests

REST requires requests to contain the information needed to understand them.

The server should not depend on hidden conversational state such as:

EXAMPLE / FLOW

```
Request 1:
"Select Gift 100"

Request 2:
"Now complete the selected Gift"
```

A better HTTP request is:

EXAMPLE / FLOW

```
POST /api/gifts/100/complete
```

The target Gift is explicit.

Stateless does **not** mean the application has no state.

Obviously your system stores:

EXAMPLE / FLOW

```
Gifts

Constituents

Campaigns

Funds
```

Stateless means:

> The server should not require undocumented per-client conversation state between requests in order to interpret the next request.

7. Resources

REST APIs are usually designed around **resources**.

Examples from the Fundraising Platform:

EXAMPLE / FLOW

```
Constituent

Gift

Campaign

Fund

GiftAllocation

Interaction
```

A URI identifies a resource or collection.

Examples:

EXAMPLE / FLOW

```
/api/gifts

/api/gifts/125

/api/campaigns/10

/api/constituents/200

/api/gifts/125/allocations
```

8. Resource URI Design

Good resource URIs usually use nouns:

EXAMPLE / FLOW

```
/api/gifts

/api/campaigns

/api/funds
```

Avoid designing every route like an RPC method:

EXAMPLE / FLOW

```
/api/GetGift

/api/CreateGift

/api/DeleteGift

/api/UpdateCampaign
```

The HTTP method already communicates much of the operation.

For example:

EXAMPLE / FLOW

```
GET /api/gifts/125

POST /api/gifts

DELETE /api/gifts/125
```

9. Plural Resource Names

A consistent convention is:

EXAMPLE / FLOW

```
/api/gifts

/api/campaigns

/api/funds

/api/constituents
```

rather than mixing:

EXAMPLE / FLOW

```
/api/gift

/api/campaigns

/api/fund
```

Consistency matters more than whether singular or plural is philosophically superior.

For normal REST APIs, plural collection names are common and readable.

10. Collection vs Individual Resource

Collection:

EXAMPLE / FLOW

```
/api/gifts
```

Individual resource:

EXAMPLE / FLOW

```
/api/gifts/125
```

This maps naturally to operations:

EXAMPLE / FLOW

```
GET /api/gifts
    →
Get collection

GET /api/gifts/125
    →
Get one Gift

POST /api/gifts
    →
Create a Gift
```

Diagram 1 — Resource-Oriented API

EXAMPLE / FLOW

```
/api/gifts
   │
   ├── GET
```

```
   │      ↓
   │   Search/List Gifts
   │
   └── POST
         ↓
       Create Gift


/api/gifts/125
   │
   ├── GET
   │     ↓
   │   Retrieve Gift
   │
   ├── PUT
   │     ↓
   │   Replace/update representation
   │
   └── DELETE
         ↓
       Delete Gift
```

WEB API / REST

PART 1 — HTTP METHODS

11. GET

`GET` retrieves a representation.

Example:

EXAMPLE / FLOW

```
GET /api/gifts/125
```

Response:

EXAMPLE / FLOW

```
200 OK
```

with:

C#

```
{
  "giftId": 125,
  "amount": 500.00,
  "status": "Completed"
}
```

GET should not be designed to request a state-changing operation.

HTTP defines safe methods as essentially read-only from the client's requested semantics.

12. Do Not Change Business State with GET

Bad:

EXAMPLE / FLOW

```
GET /api/gifts/125/complete
```

This causes an important state transition using a safe/read method.

Automated systems such as:

EXAMPLE / FLOW
```
crawlers

prefetchers

caches

link checkers
```

can issue GET requests.

Use a state-changing method instead:

EXAMPLE / FLOW
```
POST /api/gifts/125/complete
```

13. POST

POST usually submits data for processing or creates a subordinate/new resource.

Example:

EXAMPLE / FLOW
```
POST /api/gifts
```

Request:

C#
```
{
  "constituentId": 100,
  "campaignId": 5,
  "amount": 250.00
}
```

The server determines the new Gift ID.

Successful creation typically returns:

EXAMPLE / FLOW
```
201 Created
```

with a:

EXAMPLE / FLOW
```
Location
```

header pointing at the created resource.

RFC 9110 defines `201 Created` as indicating that the request created one or more resources and identifies the primary resource through `Location` or the target URI.

14. POST Is Not Automatically Idempotent

Suppose:

EXAMPLE / FLOW
```
POST /api/gifts
```

is sent twice.

Without protection, you might create:

```
EXAMPLE / FLOW
Gift 501

Gift 502
```

Therefore ordinary POST creation is usually not naturally idempotent.

For operations vulnerable to network retries, APIs sometimes implement:

```
EXAMPLE / FLOW
Idempotency-Key
```

or another business deduplication mechanism.

We will return to this later.

15. PUT

PUT conceptually creates or replaces the state of the resource identified by the request URI.

Example:

```
EXAMPLE / FLOW
PUT /api/campaigns/10
```

Request:

```
C#
{
  "name": "Annual Giving 2027",
  "startDate": "2027-01-01",
  "endDate": "2027-12-31"
}
```

The important semantic idea is:

```
EXAMPLE / FLOW
client knows the target URI
```

and sends the desired representation.

16. PUT Is Idempotent

Suppose this request is repeated:

```
EXAMPLE / FLOW
PUT /api/campaigns/10
```

with the same body.

The intended final state should remain the same.

HTTP explicitly defines PUT and DELETE, along with safe methods, as idempotent methods.

Idempotency means:

Multiple identical requests have the same intended server-side effect as one re

It does **not** mean the server performs zero internal side effects.

Logs and audit entries may still differ.

17. PUT vs PATCH

A useful mental distinction is:

EXAMPLE / FLOW

```
PUT
    →
Replace/update the resource representation as a whole

PATCH
    →
Apply partial modifications
```

Example PUT:

C#

```
{
  "name": "Annual Campaign",
  "active": true,
  "description": "Annual fundraising campaign"
}
```

Example conceptual PATCH:

C#

```
{
  "active": false
}
```

PATCH is useful when clients should change only selected fields.

Do not blindly map patch input directly onto a domain entity if that bypasses business rules.

18. PATCH and Domain Behavior

Bad:

EXAMPLE / FLOW

```
PATCH request
    ↓
reflection sets Gift.Status
    ↓
domain invariant bypassed
```

For example, a client should not be able to send:

C#

```
{
  "status": "Completed"
}
```

and bypass:

```
gift.Complete();
```

A partial update API still needs to respect:

EXAMPLE / FLOW

```
domain invariants

authorization

state transition rules
```

19. DELETE

DELETE requests removal of the target resource.

Example:

EXAMPLE / FLOW

```
DELETE /api/campaigns/10
```

A successful synchronous DELETE commonly returns:

EXAMPLE / FLOW

```
204 No Content
```

when there is no response representation.

HTTP defines 204 as successful completion with no additional response content.

20. DELETE Is Idempotent

Suppose:

EXAMPLE / FLOW

```
DELETE /api/campaigns/10
```

is sent twice.

After the first successful request:

EXAMPLE / FLOW

```
Campaign 10 no longer exists
```

The second request doesn't create another deletion.

The intended final state remains:

EXAMPLE / FLOW

```
Campaign 10 absent
```

Therefore DELETE is idempotent in HTTP semantics.

The second request may return:

EXAMPLE / FLOW

```
404
```

and the method can still be considered idempotent.

Idempotency refers to intended effect, not identical response status.

21. Safe vs Idempotent

These concepts are different.

Safe:

EXAMPLE / FLOW

```
client doesn't request state change
```

Idempotent:

EXAMPLE / FLOW

```
repeating same request has same intended final effect
```

Examples:

Method	Safe	Idempotent
GET	Yes	Yes
HEAD	Yes	Yes
POST	No	Usually no
PUT	No	Yes
DELETE	No	Yes
PATCH	No	Not guaranteed

RFC 9110 defines safe and idempotent semantics separately.

PART 2 — HTTP STATUS CODES

22. Why Status Codes Matter

Clients should not parse random response strings like:

C#

```
{
  "message": "Something happened"
}
```

to understand whether an operation succeeded.

HTTP already gives standardized status semantics.

Good API design uses:

EXAMPLE / FLOW

```
status code
+
headers
+
optional body
```

consistently.

23. 200 OK

Use for a successful request when a response representation is returned.

Example:

EXAMPLE / FLOW
```
GET /api/gifts/125
```

Response:

EXAMPLE / FLOW
```
200 OK
Content-Type: application/json
```

24. 201 Created

Use when a request successfully created a resource.

Example:

EXAMPLE / FLOW
```
POST /api/gifts
```

Response:

EXAMPLE / FLOW
```
201 Created
Location: /api/gifts/501
```

RFC 9110 explicitly associates 201 with resource creation.

25. 202 Accepted

Use when:

EXAMPLE / FLOW
```
request accepted
```

but processing will happen asynchronously.

Example:

EXAMPLE / FLOW
```
POST /api/gift-exports
```

Response:

EXAMPLE / FLOW
```
202 Accepted
```

The API might return:

C#
```
{
  "jobId": "abc123",
  "status": "Queued"
}
```

This is appropriate for:

EXAMPLE / FLOW

```
large exports

bulk imports

long-running reporting

asynchronous workflow initiation
```

26. 204 No Content

Use when the operation succeeded and no body is needed.

Examples:

EXAMPLE / FLOW

```
DELETE completed

Update completed and response body unnecessary

Command completed successfully
```

RFC 9110 defines 204 precisely as successful processing with no additional content to send.

27. 400 Bad Request

Typical causes:

EXAMPLE / FLOW

```
Malformed request

Invalid request DTO

Model binding failure

Validation failure
```

For `[ApiController]` controllers, invalid model state automatically produces an HTTP 400 response by default, with `ValidationProblemDetails`.

28. 401 Unauthorized

Means the request lacks valid authentication credentials.

Despite the name:

EXAMPLE / FLOW

```
401 Unauthorized
```

is primarily an **authentication** response.

Example:

EXAMPLE / FLOW

```
missing or invalid bearer token
```

29. 403 Forbidden

Means:

EXAMPLE / FLOW
```
caller is authenticated
```

but:

EXAMPLE / FLOW
```
caller is not permitted to perform operation
```

Example:

EXAMPLE / FLOW
```
User can view Gifts
but cannot refund Gifts
```

30. 404 Not Found

Use when the requested resource is unavailable/not found.

Example:

EXAMPLE / FLOW
```
GET /api/gifts/999999
```

Response:

EXAMPLE / FLOW
```
404 Not Found
```

31. 405 Method Not Allowed

Suppose the route supports:

EXAMPLE / FLOW
```
GET /api/gifts/125
```

but client sends:

EXAMPLE / FLOW
```
DELETE /api/gifts/125
```

and DELETE isn't supported.

The framework may return:

EXAMPLE / FLOW
```
405 Method Not Allowed
```

The path can be valid while the HTTP method is not.

32. 409 Conflict

Use when the request conflicts with the current resource state.

Potential Fundraising examples:

EXAMPLE / FLOW

```
Gift already completed

Duplicate external transaction reference

Campaign cannot be closed while processing remains

Optimistic concurrency conflict
```

RFC 9110 defines 409 as a request conflict with the current state of the target resource.

33. 412 Precondition Failed

Useful with conditional requests.

Example:

EXAMPLE / FLOW

```
PUT /api/gifts/125
If-Match: "version-7"
```

If the current server representation has:

EXAMPLE / FLOW

```
version-8
```

the request precondition fails.

Response:

EXAMPLE / FLOW

```
412 Precondition Failed
```

HTTP defines failed `If-Match` state-changing preconditions as producing 412.

This can be used to prevent lost updates.

34. 415 Unsupported Media Type

Example:

EXAMPLE / FLOW

```
POST /api/gifts
Content-Type: application/xml
```

but the API only accepts JSON.

A suitable result may be:

EXAMPLE / FLOW

```
415 Unsupported Media Type
```

35. 422 Unprocessable Content

Some API designs use 422 when:

EXAMPLE / FLOW

```
request syntax is valid
```

WEB API / REST

but:

EXAMPLE / FLOW

```
the semantic request cannot be processed
```

For example:

EXAMPLE / FLOW

```
Campaign exists
but business state prevents operation
```

However, ASP.NET Core's `[ApiController]` model validation behavior defaults to 400 for model-state validation.

Do not introduce 422 simply because another API uses it.

Choose a consistent contract.

36. Status-Code Design Rule

Do not return:

EXAMPLE / FLOW

```
200 OK
```

with:

C#

```
{
  "success": false,
  "error": "Gift not found"
}
```

for ordinary HTTP errors.

Use the protocol.

For example:

EXAMPLE / FLOW

```
404 Not Found
```

with a structured Problem Details body.

PART 3 — CONTROLLER-BASED APIs

37. ControllerBase

Controller-based Web APIs usually derive from:

C#

```
ControllerBase
```

Example:

```C#
[ApiController]
[Route("api/gifts")]
public sealed class GiftsController :
    ControllerBase
{
}
```

`ControllerBase` provides helpers such as:

```C#
Ok(...)

Created(...)

CreatedAtAction(...)

NoContent()

BadRequest(...)

NotFound()

Conflict(...)

Problem(...)
```

Controller-based API support is part of current ASP.NET Core.

38. Controller vs ControllerBase

`ControllerBase`:

```EXAMPLE / FLOW
Web API focused
```

`Controller`:

```EXAMPLE / FLOW
inherits ControllerBase
+
adds view-related MVC functionality
```

For a JSON Web API:

```C#
ControllerBase
```

is normally the appropriate base class.

39. ApiController Attribute

Typical controller:

```C#
[ApiController]
[Route("api/gifts")]
public sealed class GiftsController :
    ControllerBase
```

```
{
}
```


`[ApiController]` enables API-specific behavior including:

EXAMPLE / FLOW

```
Attribute routing requirement

Automatic HTTP 400 responses

Binding-source inference

Multipart/form-data inference

Problem-details behavior for error status codes
```

Microsoft documents these current behaviors explicitly.

40. Automatic Validation Response

Without `[ApiController]`, older-style actions may need:

C#

```
if (!ModelState.IsValid)
{
    return BadRequest(
        ModelState);
}
```

With:

C#

```
[ApiController]
```

invalid model state automatically produces:

EXAMPLE / FLOW

```
400 Bad Request
```

and usually:

EXAMPLE / FLOW

```
ValidationProblemDetails
```

before your action executes.

41. Example ValidationProblemDetails

Conceptually:

C#

```
{
  "type": "https://example.com/problems/validation",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "amount": [
      "Amount must be greater than zero."
    ]
```

```
  }
}
```


This is much better than returning unrelated error formats from every endpoint.

42. Binding Source Inference

`[ApiController]` can infer where parameters should come from.

The available explicit binding attributes include:

EXAMPLE / FLOW

```
[FromBody]

[FromRoute]

[FromQuery]

[FromHeader]

[FromForm]

[FromServices]
```

Microsoft documents these request sources and `[ApiController]` inference behavior.

PART 4 — MODEL BINDING

43. What Is Model Binding?

HTTP values arrive as:

EXAMPLE / FLOW

```
route strings

query-string values

headers

form data

JSON body
```

Your C# code wants:

C#

```
long

decimal

bool

DateTime

record

class
```

Model binding converts request data into .NET parameters and objects.

Microsoft describes model binding as automatically retrieving request values, converting them to .NET types, and supplying them to actions.

44. FromRoute

Route:

EXAMPLE / FLOW
```
/api/gifts/125
```

Action:

C#
```
[HttpGet("{giftId:long}")]
public async Task<ActionResult<GiftResponse>>
    Get(
        [FromRoute] long giftId,
        CancellationToken cancellationToken)
{
}
```

Value:

EXAMPLE / FLOW
```
giftId = 125
```

45. FromQuery

Request:

EXAMPLE / FLOW
```
GET /api/gifts?page=2&pageSize=50&status=Completed
```

Action:

C#
```
public async Task<ActionResult<PagedResult<GiftListItem>>>
    Search(
        [FromQuery] GiftSearchRequest request,
        CancellationToken cancellationToken)
{
}
```

Query parameters are appropriate for things such as:

EXAMPLE / FLOW
```
Filtering

Sorting

Search

Pagination

Optional read parameters
```

46. FromBody

Request:

EXAMPLE / FLOW

```
POST /api/gifts
Content-Type: application/json
```

Body:

C#

```
{
  "constituentId": 100,
  "amount": 250
}
```

Action:

C#

```
public async Task<ActionResult<GiftResponse>>
    Create(
        [FromBody] CreateGiftRequest request,
        CancellationToken cancellationToken)
{
}
```

`[ApiController]` can infer `[FromBody]` for appropriate complex action parameters.

47. FromHeader

Example:

EXAMPLE / FLOW

```
Idempotency-Key: 3a0724d0-...
```

Action:

C#

```
public Task<IActionResult> Create(
    [FromHeader(Name = "Idempotency-Key")]
        string? idempotencyKey)
```

Headers are appropriate for:

EXAMPLE / FLOW

```
Protocol metadata

Conditional requests

Correlation identifiers

Idempotency metadata

Authentication-related protocol values
```

not ordinary domain payload fields.

48. FromServices

Example:

```C#
public IActionResult Example(
    [FromServices]
        IClock clock)
```

ASP.NET Core can resolve services from DI.

However, constructor injection is usually clearer for controller-wide dependencies.

Use action-level service injection when the dependency genuinely applies only to that action.

49. Model Binding Conversion Failure

Suppose action expects:

```C#
long giftId
```

and the model binder receives:

```EXAMPLE / FLOW
abc
```

Binding fails.

Model-state errors record conversion failures.

Microsoft explicitly distinguishes:

```EXAMPLE / FLOW
model binding errors
    →
conversion/input acquisition failures
```

from:

```EXAMPLE / FLOW
validation errors
    →
bound value violates validation rules
```

PART 5 — REQUEST VALIDATION

50. DataAnnotations

A request DTO can use:

```C#
public sealed class CreateGiftRequest
{
    [Range(
        0.01,
        1_000_000)]
    public decimal Amount
    {
```

```
        get;
        init;
    }

    [Required]
    public string ExternalReference
    {
        get;
        init;
    } = string.Empty;
}
```

Microsoft documents built-in attributes including:

EXAMPLE / FLOW

```
Required

Range

StringLength

RegularExpression

EmailAddress

Phone

Compare

Url
```

51. Validation Happens After Binding

Conceptually:

EXAMPLE / FLOW

```
HTTP request
    ↓
Model Binding
    ↓
.NET request object
    ↓
Model Validation
    ↓
Action
```

If `[ApiController]` is active and validation fails:

EXAMPLE / FLOW

```
Action does not run
    ↓
400 ValidationProblemDetails
```

52. Request Validation vs Domain Validation

This distinction is essential.

Request validation:

EXAMPLE / FLOW

```
Amount field required

Maximum string length

Date format valid

Page size between 1 and 100
```

Domain validation:

EXAMPLE / FLOW

```
Gift cannot complete unless allocations total Gift amount

Inactive Fund cannot receive allocation

Campaign cannot accept Gift after business-defined closure
```

Do not put every business invariant into DTO attributes.

53. DTO Validation Is Boundary Validation

Example:

C#

```
public sealed record CreateGiftRequest(
    [property: Range(
        0.01,
        1_000_000)]
    decimal Amount);
```

This protects:

EXAMPLE / FLOW

```
HTTP boundary
```

But Domain still needs:

C#

```
Gift.Create(...);
```

to enforce its own rules.

Why?

Because the Domain may also be called from:

EXAMPLE / FLOW

```
background worker

message handler

unit test

another application use case
```

not only this one HTTP endpoint.

54. Non-Nullable Reference Types and Validation

WEB API / REST

With nullable reference types enabled, MVC treats non-nullable bound properties/parameters as implicitly required in many model-validation scenarios.

Microsoft documents that non-nullable bound properties are treated similarly to having an implicit `[Required(AllowEmptyStrings = true)]` rule.

Still, explicit attributes can be useful when:

EXAMPLE / FLOW

```
you want a specific error message

the business contract should be obvious

string constraints go beyond nullability
```

PART 6 — DTO DESIGN

55. Do Not Expose EF/Domain Entities Directly

Bad:

C#

```
[HttpGet("{giftId:long}")]
public async Task<Gift> Get(
    long giftId)
{
    return await dbContext.Gifts
        .Include(...)
        .SingleAsync(...);
}
```

Problems:

EXAMPLE / FLOW

```
Internal fields may leak

Navigation properties may serialize unexpectedly

API becomes coupled to persistence shape

Circular references can appear

Changing domain model breaks client contract

Over-posting becomes easier
```

Use API DTOs.

56. Response DTO

Example:

C#

```
public sealed record GiftResponse(
    long GiftId,
    decimal Amount,
    GiftStatus Status,
```

```
    long ConstituentId,
    long? CampaignId);
```


Now the API deliberately controls its public representation.

57. Separate Request and Response DTOs

Create request:

C#

```
public sealed record CreateGiftRequest(
    long ConstituentId,
    long? CampaignId,
    decimal Amount);
```

Response:

C#

```
public sealed record GiftResponse(
    long GiftId,
    long ConstituentId,
    long? CampaignId,
    decimal Amount,
    string Status);
```

Notice:

EXAMPLE / FLOW

```
Create request
    →
no GiftId

Response
    →
contains GiftId

Response
    →
contains system status

Create request
    →
doesn't let client set status
```

This protects API semantics.

58. Update DTO Should Also Be Deliberate

Bad:

C#

```
public sealed class GiftDto
{
    public long GiftId { get; set; }

    public long TenantId { get; set; }

    public decimal Amount { get; set; }
```

```
    public GiftStatus Status { get; set; }
}
```

WEB API / REST

used for:

EXAMPLE / FLOW

```
create

update

response
```

Clients can now potentially send fields they should never control.

Prefer purpose-specific contracts.

PART 7 — CONTROLLER ACTION RETURN TYPES

59. Specific Type

Example:

C#

```
[HttpGet]
public IReadOnlyList<GiftListItem>
    Get()
{
    return ...;
}
```

A specific type is useful when:

EXAMPLE / FLOW

```
only one successful response shape exists

no alternate ActionResult is needed
```

Current ASP.NET Core controller actions support specific return types, `IActionResult`, `ActionResult<T>`, and HTTP result types.

60. IActionResult

Use when an action may return several HTTP result types.

Example:

C#

```
[HttpDelete("{giftId:long}")]
public async Task<IActionResult> Delete(
    long giftId,
    CancellationToken cancellationToken)
{
    bool deleted =
        await _service.DeleteAsync(
            giftId,
            cancellationToken);

    if (!deleted)
    {
        return NotFound();
```

```
    }

    return NoContent();
}
```

Possible results:

```
EXAMPLE / FLOW
404

204
```

Microsoft describes `IActionResult` as appropriate when multiple action-result types are possible.

61. ActionResult<T>

This is often excellent for GET endpoints.

Example:

```
C#
[HttpGet("{giftId:long}")]
public async Task<ActionResult<GiftResponse>>
    Get(
        long giftId,
        CancellationToken cancellationToken)
{
    GiftResponse? gift =
        await _queries.GetAsync(
            giftId,
            cancellationToken);

    if (gift is null)
    {
        return NotFound();
    }

    return gift;
}
```

Possible values:

```
EXAMPLE / FLOW
GiftResponse
or
ActionResult such as NotFound()
```

Microsoft notes that `ActionResult<T>` can return either `T` or an `ActionResult`, and the successful response type can be inferred from `T` for API metadata.

62. CreatedAtAction

For creation:

```
C#
[HttpPost]
public async Task<ActionResult<GiftResponse>>
    Create(
        CreateGiftRequest request,
        CancellationToken cancellationToken)
{
```

```
    GiftResponse result =
        await _service.CreateAsync(
            request,
            cancellationToken);

    return CreatedAtAction(
        nameof(Get),
        new
        {
            giftId =
                result.GiftId
        },
        result);
}
```

WEB API / REST

This communicates:

```
EXAMPLE / FLOW
201 Created
+
Location header
+
response representation
```

ASP.NET Core's Web API documentation uses `CreatedAtAction` for exactly this create-resource pattern.

63. Why CreatedAtAction Is Better Than Ok for Creation

Bad:

```
C#
return Ok(
    createdGift);
```

This returns:

```
EXAMPLE / FLOW
200
```

but loses the stronger protocol semantics that:

```
EXAMPLE / FLOW
new resource was created
```

Better:

```
C#
return CreatedAtAction(...);
```

Clients can then discover the new resource URI from `Location`.

64. IAsyncEnumerable<T>

Controller actions can return:

```
C#
IAsyncEnumerable<T>
```

for asynchronous iteration/streaming scenarios.

Current ASP.NET Core documentation notes that MVC buffers `IEnumerable<T>` results, whereas `IAsyncEnumerable<T>` can be streamed with the `System.Text.Json` formatter.

WEB API / REST

Example:

C#
```
[HttpGet("stream")]
public async IAsyncEnumerable<GiftListItem>
    StreamGifts(
        [EnumeratorCancellation]
        CancellationToken cancellationToken)
{
    await foreach (
        GiftListItem gift
        in _queries.StreamAsync(
            cancellationToken))
    {
        yield return gift;
    }
}
```

Use only where streaming actually benefits the client.

For normal paged search endpoints, pagination is usually simpler.

PART 8 — MINIMAL APIs

65. Minimal API Example

C#
```
app.MapGet(
    "/api/gifts/{giftId:long}",
    async (
        long giftId,
        GiftQueries queries,
        CancellationToken cancellationToken) =>
    {
        GiftResponse? result =
            await queries.GetAsync(
                giftId,
                cancellationToken);

        return result is null
            ? Results.NotFound()
            : Results.Ok(
                result);
    });
```

Minimal APIs are first-class ASP.NET Core API endpoints.

66. Minimal API Return Values

Minimal API handlers can return:

EXAMPLE / FLOW
```
IResult

string

arbitrary T
```

```
Task<T>
  ✳
ValueTask<T>
```

WEB API / REST

ASP.NET Core serializes ordinary object return values as JSON and uses `IResult` implementations for explicit HTTP behavior.

67. Results

Example:

```
C#
return Results.Ok(
    gift);
```

or:

```
C#
return Results.NotFound();
```

`Results` helpers return `IResult`.

68. TypedResults

Example:

```
C#
return TypedResults.Ok(
    gift);
```

`TypedResults` returns concrete strongly typed HTTP result objects.

Current Microsoft guidance notes benefits including:

```
EXAMPLE / FLOW
stronger typing

better unit-test ergonomics

automatic response metadata for OpenAPI
```

69. Results<T1, T2>

For multiple typed result possibilities:

```
C#
static async Task<
    Results<
        Ok<GiftResponse>,
        NotFound>>
    GetGift(...)
{
}
```

ASP.NET Core supports generic `Results<T1, ...>` unions for Minimal APIs.

They preserve endpoint metadata and allow compile-time checking that only declared result types are returned.

70. Controller vs Minimal API

Controller advantages:

```
EXAMPLE / FLOW
Familiar MVC organization

Attributes

Filters

Conventions

Rich controller ecosystem

Good fit for large traditional APIs
```

Minimal API advantages:

```
EXAMPLE / FLOW
Less ceremony

Feature-local route definitions

Typed HTTP results

Good fit for vertical slices

Easy route grouping
```

There is no rule that:

```
EXAMPLE / FLOW
Minimal APIs = small projects only
```

or:

```
EXAMPLE / FLOW
Controllers = enterprise only
```

Both can support production systems.

Choose based on organization, team conventions, required framework features, and readability.

PART 9 — CONTENT NEGOTIATION

71. What Is Content Negotiation?

Client may send:

```
EXAMPLE / FLOW
Accept: application/json
```

This tells the server which response representations are acceptable.

ASP.NET Core MVC uses output formatters and `ObjectResult` to perform content negotiation for controller responses.

72. Default Response Formats

ASP.NET Core controller Web APIs support JSON by default and include common media types such as:

EXAMPLE / FLOW

```
application/json

text/json

text/plain
```

depending on the return type/formatter behavior.

For most REST APIs:

EXAMPLE / FLOW

```
application/json
```

is the primary representation.

73. Accept Header

Example:

EXAMPLE / FLOW

```
GET /api/gifts/125
Accept: application/json
```

ASP.NET Core examines the `Accept` header and attempts to find an appropriate formatter.

If none is available, behavior depends on configuration.

For example:

C#

```
builder.Services
    .AddControllers(
        options =>
        {
            options.ReturnHttpNotAcceptable = true;
        });
```

can cause:

EXAMPLE / FLOW

```
406 Not Acceptable
```

when a requested representation cannot be produced.

74. Content-Type vs Accept

Do not confuse them.

`Content-Type`:

EXAMPLE / FLOW

```
What format is the request/response body currently in?
```

`Accept`:

EXAMPLE / FLOW

```
What response formats does the client accept?
```

WEB API / REST

Example:

EXAMPLE / FLOW

```
POST /api/gifts
Content-Type: application/json
Accept: application/json
```

75. Consumes

Controller endpoint:

C#

```
[Consumes(
    "application/json")]
[HttpPost]
public IActionResult Create(
    CreateGiftRequest request)
{
}
```

This documents/constrains accepted request content types.

ASP.NET Core's Web API controller documentation includes `[Consumes]` as a supported API attribute.

76. Produces

You can describe produced media types:

C#

```
[Produces(
    "application/json")]
```

and response metadata:

C#

```
[ProducesResponseType(
    typeof(GiftResponse),
    StatusCodes.Status200OK)]
```

These can improve API documentation and tooling.

PART 10 — JSON SERIALIZATION

77. System.Text.Json

Modern ASP.NET Core uses:

EXAMPLE / FLOW

```
System.Text.Json
```

as the normal built-in JSON serializer.

Typical request:

```
C#
{
  "giftId": 125,
  "totalAmount": 250.0
}
```

maps to C# DTOs and vice versa.

78. Configure JSON

Example:

```
C#
builder.Services
    .AddControllers()
    .AddJsonOptions(
        options =>
        {
            options.JsonSerializerOptions
                .WriteIndented = false;
        });
```

You can configure settings such as:

```
EXAMPLE / FLOW
Naming policy

Enum conversion

Converters

Reference handling

Number handling
```

Do not change serialization conventions casually after clients depend on them.

JSON shape is part of your API contract.

79. Enum Serialization

Suppose:

```
C#
public enum GiftStatus
{
    Pending,
    Completed,
    Cancelled
}
```

You must deliberately decide whether the API exposes:

```
C#
{
  "status": 1
}
```

or:

```C#
{
  "status": "Completed"
}
```

String representations are often easier for clients to understand, but they also create a public string contract.

Changing enum member names can then become a breaking change.

PART 11 — PROBLEM DETAILS

80. Standard Error Shape

Instead of every endpoint returning different errors:

```C#
{
  "error": "..."
}
```

```C#
{
  "message": "..."
}
```

```C#
{
  "errors": "..."
}
```

use a consistent Problem Details strategy.

ASP.NET Core provides support for `ProblemDetails`, and controller APIs automatically transform many client errors into Problem Details.

81. ProblemDetails Example

```C#
{
  "type": "https://api.example.com/problems/gift-not-found",
  "title": "Gift was not found.",
  "status": 404,
  "detail": "Gift 125 could not be located.",
  "instance": "/api/gifts/125",
  "traceId": "00-..."
}
```

Useful fields include:

EXAMPLE / FLOW

```
type

title

status

detail
```

```
instance

extensions
```

82. Do Not Leak Internal Details

Bad:

```C#
{
  "detail":
    "SqlException: Invalid column fundraising.Gift.X at C:\\src\\..."
}
```

Production responses should not expose:

```
Stack traces

SQL

Secrets

Internal file paths

Sensitive identifiers

Infrastructure internals
```

Use server logs for technical details.

PART 12 — PAGINATION

83. Never Return an Unbounded Collection by Default

Bad:

```
GET /api/gifts
```

returns:

```
4,000,000 Gifts
```

Problems:

```
Huge SQL query

High memory

Large JSON serialization

Network bandwidth

Slow clients
```

```
Long database locks/resources
*
Timeout risk
```

WEB API / REST

Use pagination.

84. Offset Pagination

Request:

EXAMPLE / FLOW

```
GET /api/gifts?page=3&pageSize=50
```

Query:

C#

```
var gifts =
    await dbContext.Gifts
        .AsNoTracking()
        .OrderByDescending(
            gift =>
                gift.GiftId)
        .Skip(
            (page - 1)
            *
            pageSize)
        .Take(
            pageSize)
        .Select(...)
        .ToListAsync(
            cancellationToken);
```

Simple and intuitive.

85. Paged Response

Example:

C#

```
public sealed record PagedResult<T>(
    IReadOnlyList<T> Items,
    int Page,
    int PageSize,
    long TotalCount);
```

Response:

C#

```
{
  "items": [],
  "page": 3,
  "pageSize": 50,
  "totalCount": 12482
}
```

86. Limit Page Size

Never trust:

EXAMPLE / FLOW

```
?pageSize=10000000
```

WEB API / REST

A request DTO can validate:

C#

```
public sealed record GiftSearchRequest
{
    [Range(
        1,
        200)]
    public int PageSize
    {
        get;
        init;
    } = 50;
}
```

This is a good HTTP-boundary validation rule.

87. Cursor/Keyset Pagination

Offset pagination can become inefficient for deep pages and can behave awkwardly when records are inserted or deleted during paging.

Keyset/cursor style:

EXAMPLE / FLOW

```
GET /api/gifts?afterGiftId=50000&pageSize=50
```

Query:

C#

```
.Where(
    gift =>
        gift.GiftId < afterGiftId)
.OrderByDescending(
    gift =>
        gift.GiftId)
.Take(
    pageSize)
```

can scale better in appropriate cases.

Choose based on access patterns.

PART 13 — FILTERING, SORTING, SEARCH

88. Filtering

Example:

EXAMPLE / FLOW

```
GET /api/gifts?status=Completed&campaignId=10
```

Filtering belongs naturally in query parameters.

89. Sorting

Example:

EXAMPLE / FLOW
```
GET /api/gifts?sort=-giftDate
```

or explicit:

EXAMPLE / FLOW
```
GET /api/gifts?sortBy=giftDate&sortDirection=desc
```

Never dynamically concatenate arbitrary client input into SQL.

Map allowed sort fields explicitly.

90. Search

Example:

EXAMPLE / FLOW
```
GET /api/constituents?search=smith
```

A general free-text search might search:

EXAMPLE / FLOW
```
Name

Email

External reference
```

The exact semantics should be documented.

Do not let:

EXAMPLE / FLOW
```
search
```

mean something completely different on every endpoint.

91. Query DTO

C#
```
public sealed record GiftSearchRequest
{
    public GiftStatus? Status
    {
        get;
        init;
    }

    public long? CampaignId
    {
        get;
        init;
    }

    public int Page
```

```
    {
        get;
        init;
    } = 1;

    [Range(
        1,
        200)]
    public int PageSize
    {
        get;
        init;
    } = 50;

    public string? SortBy
    {
        get;
        init;
    }
}
```

This is cleaner than a controller with 15 unrelated parameters.

PART 14 — IDEMPOTENCY

92. Why Idempotency Matters

Suppose a client sends:

EXAMPLE / FLOW

```
POST /api/gifts
```

The server creates Gift 501.

Before the client receives the response:

EXAMPLE / FLOW

```
network connection fails
```

Client doesn't know whether Gift was created.

It retries.

Without protection:

EXAMPLE / FLOW

```
Gift 501
Gift 502
```

may both be created.

93. Idempotency Key Concept

Client sends:

EXAMPLE / FLOW

```
Idempotency-Key: 6f0a032c-...
```

Server records:

EXAMPLE / FLOW

```
Key
Request identity/hash
Result/resource
```

When the same key arrives again:

EXAMPLE / FLOW

```
return previous result
```

instead of duplicating the business operation.

This is particularly useful for:

EXAMPLE / FLOW

```
payments

Gift processing

financial operations

external integration commands
```

94. Idempotency Is a Business Requirement Too

It is not enough to say:

EXAMPLE / FLOW

```
HTTP POST isn't idempotent
```

Financial systems frequently need deduplication regardless of HTTP method semantics.

For your Fundraising Platform, useful unique business identifiers may include:

EXAMPLE / FLOW

```
Payment processor transaction ID

External Gift reference

Import batch + external record ID

Idempotency key
```

These often belong in:

EXAMPLE / FLOW

```
database uniqueness

application logic

integration handling
```

together.

PART 15 — CONCURRENCY AND CONDITIONAL REQUESTS

95. Lost Update Problem

Two users retrieve:

EXAMPLE / FLOW

```
Gift version 7
```

User A changes Campaign.

User B changes amount/status-related information.

If B saves later without checking the version:

EXAMPLE / FLOW

```
A's update may be overwritten
```

This is the lost-update problem.

96. EF Core Concurrency + HTTP

Suppose SQL Server uses:

EXAMPLE / FLOW

```
rowversion
```

Domain/read response exposes a safe version token.

Server returns:

EXAMPLE / FLOW

```
ETag: "abc123"
```

Client updates:

EXAMPLE / FLOW

```
PUT /api/gifts/125
If-Match: "abc123"
```

Server compares the version.

If stale:

EXAMPLE / FLOW

```
412 Precondition Failed
```

The HTTP specification explicitly defines failed `If-Match` state-changing preconditions as 412.

97. 409 vs 412 for Concurrency

Both may appear in APIs.

Use:

EXAMPLE / FLOW

```
412
```

when the client explicitly supplied a failed HTTP precondition such as:

EXAMPLE / FLOW

```
If-Match
```

Use:

EXAMPLE / FLOW

```
409
```

for a general current-state conflict where HTTP conditional semantics aren't being used.

RFC 9110 defines 409 as a conflict with current resource state.

Consistency matters more than arbitrary preference.

PART 16 — CACHING

98. GET Responses Can Be Cacheable

REST includes cacheability as a core constraint.

For stable read data, HTTP supports validators such as:

EXAMPLE / FLOW

```
ETag

Last-Modified
```

Example first request:

EXAMPLE / FLOW

```
GET /api/campaigns/10
```

Response:

EXAMPLE / FLOW

```
200 OK
ETag: "v5"
```

Later:

EXAMPLE / FLOW

```
GET /api/campaigns/10
If-None-Match: "v5"
```

If unchanged:

EXAMPLE / FLOW

```
304 Not Modified
```

This avoids retransmitting the body.

99. Don't Cache Sensitive Responses Carelessly

A Gift or Constituent response may contain sensitive information.

Caching strategy must consider:

EXAMPLE / FLOW

```
Tenant

User

Authorization

Shared proxy behavior

Browser cache

Data freshness
```

Performance optimization must never weaken data isolation.

PART 17 — API VERSIONING

100. Why APIs Need Versioning Strategy

Clients may depend on your contract for years.

Suppose response changes from:

C#

```
{
  "amount": 100
}
```

to:

C#

```
{
  "amount": {
    "value": 100,
    "currency": "USD"
  }
}
```

Existing clients may break.

Therefore APIs need a compatibility strategy.

101. Versioning Approaches

Common approaches include:

EXAMPLE / FLOW

```
URI versioning

/api/v1/gifts

/api/v2/gifts
```

Query parameter:

EXAMPLE / FLOW

```
/api/gifts?api-version=2
```

Header:

EXAMPLE / FLOW

```
Api-Version: 2
```

Media type:

EXAMPLE / FLOW

```
application/vnd.company.gift-v2+json
```

No approach is universally correct.

102. Prefer Backward-Compatible Evolution

Not every new field requires:

EXAMPLE / FLOW

```
v2
```

Adding an optional response property is often compatible for tolerant clients.

Breaking changes may include:

EXAMPLE / FLOW

```
Removing property

Changing property type

Changing meaning of property

Changing URL semantics

Making optional field required

Changing enumeration contract incompatibly
```

Version only when necessary.

103. Database Version Is Not API Version

Do not connect:

EXAMPLE / FLOW

```
Database schema v27
```

to:

EXAMPLE / FLOW

```
API v27
```

The API contract and database schema evolve independently.

You may change 15 tables internally while:

EXAMPLE / FLOW

```
/api/v1/gifts
```

remains stable.

PART 18 — OPENAPI

104. What Is OpenAPI?

OpenAPI is a machine-readable description of an HTTP API.

It can describe:

EXAMPLE / FLOW

```
Routes

HTTP methods

Parameters

Request schemas

Response schemas

Status codes

Authentication requirements
```

Clients and tooling can use it for:

EXAMPLE / FLOW

```
Documentation

Client generation

Testing

API discovery
```

105. First-Party OpenAPI Support

Current ASP.NET Core includes first-party OpenAPI document generation through:

EXAMPLE / FLOW

```
Microsoft.AspNetCore.OpenApi
```

with:

C#

```
builder.Services
    .AddOpenApi();
```

and:

C#

```
app.MapOpenApi();
```

Microsoft documents `AddOpenApi` as registering document-generation services and `MapOpenApi` as exposing the generated JSON document.

106. Development-Only OpenAPI Endpoint

A typical current template uses:

```C#
if (
    app.Environment
        .IsDevelopment())
{
    app.MapOpenApi();
}
```

Microsoft's current template restricts the OpenAPI endpoint to Development by default to reduce unnecessary production exposure.

Whether your organization exposes API documentation in production is a separate security/operational decision.

107. Response Metadata

Controller example:

```C#
[ProducesResponseType(
    typeof(GiftResponse),
    StatusCodes.Status200OK)]
[ProducesResponseType(
    StatusCodes.Status404NotFound)]
[HttpGet("{giftId:long}")]
public async Task<ActionResult<GiftResponse>>
    Get(...)
```

This helps OpenAPI tooling understand:

```EXAMPLE / FLOW
200 → GiftResponse

404 → error
```

With `ActionResult<T>`, the successful response type is already inferable from `T`.

108. OpenAPI Should Match Reality

Bad documentation says:

```EXAMPLE / FLOW
POST /api/gifts
    →
201 only
```

but runtime actually produces:

```EXAMPLE / FLOW
201
400
401
403
409
500
```

Your contract/documentation should describe meaningful responses clients must handle.

OpenAPI is useful only if it stays aligned with the application.

PART 19 — CANCELLATION AND ASYNC API DESIGN

109. Request CancellationToken

Controller actions can accept:

```C#
CancellationToken cancellationToken
```

Example:

```C#
[HttpGet("{giftId:long}")]
public async Task<ActionResult<GiftResponse>>
    Get(
        long giftId,
        CancellationToken cancellationToken)
{
    GiftResponse? gift =
        await _queries.GetAsync(
            giftId,
            cancellationToken);

    return gift is null
        ? NotFound()
        : gift;
}
```

ASP.NET Core connects this to request cancellation.

If the client disconnects:

```EXAMPLE / FLOW
CancellationToken may be cancelled
```

Your I/O path should propagate it.

110. Propagate Cancellation

Good:

```EXAMPLE / FLOW
Controller token
    ↓
Application service
    ↓
Repository
    ↓
EF Core
    ↓
HttpClient
```

Example:

```C#
await dbContext.Gifts
    .ToListAsync(
        cancellationToken);
```

Do not accept a token and then ignore it throughout every expensive I/O call.

PART 20 — PERFORMANCE

111. API Performance Starts With Response Shape

Suppose list endpoint returns full Gift entities including:

EXAMPLE / FLOW

```
Constituent

Campaign

Allocations

Funds

Interactions
```

for 500 records.

Even efficient routing cannot save that design.

Prefer:

EXAMPLE / FLOW

```
Projection

Pagination

Purpose-specific DTO
```

112. Read Projection

Example:

C#

```
var result =
    await dbContext.Gifts
        .AsNoTracking()
        .Where(
            gift =>
                gift.TenantId
                ==
                tenantId)
        .Select(
            gift =>
                new GiftListItem(
                    gift.GiftId,
                    gift.TotalAmount,
                    gift.Status,
                    gift.Campaign!.Name))
        .Take(
            100)
        .ToListAsync(
            cancellationToken);
```

This is usually better than loading a giant entity graph just to return four fields.

113. Avoid N+1 API Processing

Bad:

```
EXAMPLE / FLOW
GET /api/gifts
    ↓
load 100 Gifts
    ↓
for each Gift
    ↓
query Campaign
```

Result:

```
EXAMPLE / FLOW
101 SQL queries
```

Design query shape intentionally.

114. Compression vs Huge Payloads

Response compression can reduce network transfer.

It does not justify:

```
EXAMPLE / FLOW
returning millions of unnecessary records
```

Fix:

```
EXAMPLE / FLOW
query shape

pagination

DTO size
```

first.

115. Streaming vs Pagination

Streaming:

```
EXAMPLE / FLOW
useful when client genuinely processes a sequence continuously
```

Pagination:

```
EXAMPLE / FLOW
better for typical UI list/search scenarios
```

Don't replace normal search pagination with streaming just because `IAsyncEnumerable<T>` exists.

Current ASP.NET Core supports asynchronous streaming with `IAsyncEnumerable<T>` and `System.Text.Json`.

PART 21 — SECURITY BOUNDARIES

116. DTO Validation Is Not Authorization

Suppose request contains:

```C#
{
  "campaignId": 10
}
```

Validation may prove:

```EXAMPLE / FLOW
campaignId > 0
```

It does not prove:

```EXAMPLE / FLOW
user may access Campaign 10
```

Authorization is separate.

117. Never Trust Tenant IDs from Clients

Bad:

```C#
{
  "tenantId": 999,
  "amount": 100
}
```

and server blindly uses it.

For authenticated multi-tenant systems, Tenant identity should generally come from a trusted authenticated/application context, not an arbitrary request field.

118. Over-Posting

Suppose API accepts:

```C#
Gift
```

directly.

Client sends:

```C#
{
  "amount": 100,
  "status": "Completed",
  "tenantId": 999,
  "isDeleted": false
}
```

If these properties bind directly:

EXAMPLE / FLOW

client controls internal state

WEB API / REST

Purpose-specific request DTOs prevent this class of mistake.

119. ASP.NET Core 10 Cookie Behavior for APIs

One current ASP.NET Core 10 detail: known API endpoints using cookie authentication no longer redirect unauthenticated/forbidden API requests to login/access-denied pages; API endpoints return appropriate 401/403 responses instead. Microsoft calls this out in the current Web API documentation.

Authentication and authorization are covered in depth in Topic 40.

PART 22 — COMPLETE FUNDRAISING CONTROLLER

120. Request and Response Contracts

C#

```
public sealed record CreateGiftRequest(
    long ConstituentId,
    long? CampaignId,
    decimal Amount,
    IReadOnlyList<
        CreateGiftAllocationRequest>
        Allocations);

public sealed record
    CreateGiftAllocationRequest(
        long FundId,
        decimal Amount);

public sealed record GiftResponse(
    long GiftId,
    long ConstituentId,
    long? CampaignId,
    decimal Amount,
    GiftStatus Status);
```

121. Controller

C#

```
[ApiController]
[Route("api/gifts")]
public sealed class GiftsController :
    ControllerBase
{
    private readonly
        CreateGiftService
        _createGiftService;

    private readonly
        CompleteGiftService
        _completeGiftService;

    private readonly
        GiftQueries
        _giftQueries;

    public GiftsController(
```

```
        CreateGiftService createGiftService,
        CompleteGiftService completeGiftService,
        GiftQueries giftQueries)
    {
        _createGiftService = createGiftService;

        _completeGiftService = completeGiftService;

        _giftQueries = giftQueries;
    }

    [HttpGet("{giftId:long}")]
    [ProducesResponseType(
        typeof(GiftResponse),
        StatusCodes.Status200OK)]
    [ProducesResponseType(
        StatusCodes.Status404NotFound)]
    public async Task<ActionResult<GiftResponse>>
        Get(
            long giftId,
            CancellationToken cancellationToken)
    {
        GiftResponse? gift =
            await _giftQueries.GetAsync(
                giftId,
                cancellationToken);

        if (gift is null)
        {
            return NotFound();
        }

        return gift;
    }

    [HttpPost]
    [ProducesResponseType(
        typeof(GiftResponse),
        StatusCodes.Status201Created)]
    [ProducesResponseType(
        StatusCodes.Status400BadRequest)]
    public async Task<ActionResult<GiftResponse>>
        Create(
            CreateGiftRequest request,
            CancellationToken cancellationToken)
    {
        GiftResponse gift =
            await _createGiftService
                .CreateAsync(
                    request,
                    cancellationToken);

        return CreatedAtAction(
            nameof(Get),
            new
            {
                giftId =
                    gift.GiftId
            },
            gift);
    }

    [HttpPost(
        "{giftId:long}/complete")]
    [ProducesResponseType(
```

```
        StatusCodes.Status204NoContent)]
    [ProducesResponseType(
        StatusCodes.Status404NotFound)]
    [ProducesResponseType(
        StatusCodes.Status409Conflict)]
    public async Task<IActionResult>
        Complete(
            long giftId,
            CancellationToken cancellationToken)
    {
        await _completeGiftService
            .CompleteAsync(
                giftId,
                cancellationToken);

        return NoContent();
    }
}
```

Diagram 2 — Create Gift Flow

EXAMPLE / FLOW

```
POST /api/gifts
      ↓
Model Binding
      ↓
CreateGiftRequest
      ↓
Validation
      ↓
GiftsController.Create
      ↓
CreateGiftService
      ↓
Gift.Create()
      ↓
GiftRepository
      ↓
DbContext
      ↓
SQL Server
      ↓
GiftResponse
      ↓
CreatedAtAction
      ↓
201 Created
      ↓
Location: /api/gifts/{id}
```

122. Search Endpoint

C#

```
[HttpGet]
public async Task<
    ActionResult<
        PagedResult<
            GiftListItem>>>
    Search(
        [FromQuery]
            GiftSearchRequest request,
```

```
        CancellationToken cancellationToken)
{
    PagedResult<GiftListItem> result =
        await _giftQueries.SearchAsync(
            request,
            cancellationToken);

    return Ok(
        result);
}
```

Example:

EXAMPLE / FLOW

```
GET /api/gifts?status=Completed&page=2&pageSize=50
```

PART 23 — MINIMAL API EQUIVALENT

123. Route Group

C#

```
RouteGroupBuilder gifts =
    app.MapGroup(
        "/api/gifts");
```

124. Get Gift

C#

```
gifts.MapGet(
    "/{giftId:long}",
    async Task<
        Results<
            Ok<GiftResponse>,
            NotFound>>
    (
        long giftId,
        GiftQueries queries,
        CancellationToken cancellationToken) =>
    {
        GiftResponse? gift =
            await queries.GetAsync(
                giftId,
                cancellationToken);

        return gift is null
            ? TypedResults.NotFound()
            : TypedResults.Ok(
                gift);
    });
```

Typed result unions retain response metadata and compile-time result constraints.

PART 24 — API TESTING

125. Integration Test the Contract

Using:

test:

EXAMPLE / FLOW
```
Route

Status code

Response JSON

Validation

ProblemDetails

Authentication behavior

Headers
```

Example:

C#
```
[Fact]
public async Task Get_WhenGiftDoesNotExist_Returns404()
{
    HttpResponseMessage response =
        await _client.GetAsync(
            "/api/gifts/999999");

    Assert.Equal(
        HttpStatusCode.NotFound,
        response.StatusCode);
}
```

126. Test Creation Semantics

For:

EXAMPLE / FLOW
```
POST /api/gifts
```

verify:

EXAMPLE / FLOW
```
201 Created

Location header exists

Response contains Gift ID

Resource can be retrieved through Location
```

That is more valuable than merely testing:

EXAMPLE / FLOW
```
controller method returned an object
```

127. Validation Test

Send:

```C#
{
  "amount": -100
}
```

Verify:

EXAMPLE / FLOW

```
400 Bad Request

ValidationProblemDetails

Action/application service not executed
```

`[ApiController]` provides automatic 400 model-state behavior.

PART 25 — COMMON MISTAKES

128. Verbs Everywhere in URLs

Bad:

EXAMPLE / FLOW

```
/api/CreateGift

/api/GetGift

/api/UpdateGift

/api/DeleteGift
```

Better:

EXAMPLE / FLOW

```
POST   /api/gifts

GET    /api/gifts/{id}

PUT    /api/gifts/{id}

DELETE /api/gifts/{id}
```

129. Using GET to Change State

Bad:

EXAMPLE / FLOW

```
GET /api/gifts/100/delete
```

GET is safe/read-oriented semantics.

130. Returning 200 for Every Result

Bad:

EXAMPLE / FLOW

```
200 OK

{
  "success": false,
  "errorCode": 404
}
```

Use actual HTTP status codes.

131. Returning Entity Framework Entities

Bad:

C#

```
return Ok(
    dbGiftEntity);
```

Public contracts should be deliberate DTOs.

132. One DTO for Everything

Bad:

EXAMPLE / FLOW

```
GiftDto used for:

Create

Update

Read

Database

Integration event
```

Each boundary may have different requirements.

133. Domain Logic in Controller

Bad:

C#

```
if (
    gift.Allocations.Sum(
        x =>
            x.Amount)
    ==
    gift.TotalAmount)
{
    gift.Status = GiftStatus.Completed;
}
```

Better:

```C#
gift.Complete();
```

through the Application/Domain layers.

134. Unbounded GET Endpoints

Bad:

```C#
return await dbContext.Gifts
    .ToListAsync();
```

Use:

```EXAMPLE / FLOW
pagination

filtering

projection
```

135. Accepting TenantId Blindly

Bad:

```C#
public sealed record CreateGiftRequest(
    long TenantId,
    ...);
```

if tenant identity should come from authenticated context.

136. Blind PUT/PATCH Mapping

Bad:

```C#
mapper.Map(
    request,
    gift);
```

when it can overwrite:

```EXAMPLE / FLOW
Status

TenantId

Audit fields

Domain invariants
```

137. Ignoring Idempotency in Financial Endpoints

A retry can create duplicate Gifts or charges.

Critical financial/integration endpoints should consider:

EXAMPLE / FLOW

```
idempotency keys

external reference uniqueness

deduplication
```

138. Treating DataAnnotations as Domain Rules

DTO:

C#

```
[Range(...)]
```

does not replace domain validation.

The Domain must protect itself independently.

139. Versioning Too Early

Do not create:

EXAMPLE / FLOW

```
v1

v2

v3
```

for every harmless additive change.

Use backward-compatible evolution where possible.

140. Versioning Too Late

The opposite problem:

EXAMPLE / FLOW

```
dozens of clients depend on API
```

then a breaking contract is changed in place.

Treat public contracts seriously from the beginning.

141. OpenAPI Without Accurate Response Metadata

If docs claim:

EXAMPLE / FLOW

```
200 only
```

while clients may receive:

EXAMPLE / FLOW

```
400
404
409
```

the contract is misleading.

PART 26 — BEST PRACTICES

A strong ASP.NET Core Web API should generally:

EXAMPLE / FLOW

```
Design around resources.

Use HTTP methods according to their semantics.

Keep GET safe.

Understand idempotency.

Use accurate HTTP status codes.

Return 201 + Location for created resources where appropriate.

Use DTOs rather than exposing entities.

Keep HTTP validation separate from domain invariants.

Use [ApiController] intentionally.

Use explicit binding attributes where they improve clarity.

Use ActionResult<T> for typed controller endpoints with alternate results.

Use TypedResults for strongly typed Minimal API results where useful.

Return consistent ProblemDetails.

Paginate large collections.

Validate maximum page size.

Project only required fields.

Propagate CancellationToken.

Consider idempotency for financial operations.

Use concurrency protection for conflicting updates.

Use OpenAPI and keep it accurate.

Treat JSON shape as a public contract.

Evolve APIs backward-compatibly when possible.

Avoid leaking tenant/security decisions into client-controlled data.

Integration-test the actual HTTP contract.
```

PART 27 — PRACTICAL EXERCISES

Exercise 1 — Design Gift Routes

Design routes for:

EXAMPLE / FLOW

```
List Gifts

Get Gift

Create Gift

Complete Gift

Cancel Gift

Get Gift allocations
```

A reasonable result:

EXAMPLE / FLOW

```
GET  /api/gifts

GET  /api/gifts/{giftId}

POST /api/gifts

POST /api/gifts/{giftId}/complete

POST /api/gifts/{giftId}/cancel

GET  /api/gifts/{giftId}/allocations
```

Exercise 2 — Create Endpoint

Implement:

EXAMPLE / FLOW

```
POST /api/gifts
```

returning:

EXAMPLE / FLOW

```
201 Created

Location header

GiftResponse
```

using:

C#

```
CreatedAtAction(...)
```

Exercise 3 — Automatic Validation

Add:

C#

```
[Range(
    0.01,
    1_000_000)]
```

to `Amount`.

Send:

```
C#
{
  "amount": 0
}
```

Verify `[ApiController]` returns:

```
EXAMPLE / FLOW
400 ValidationProblemDetails
```

without manually checking `ModelState`.

Exercise 4 — Binding Sources

Create endpoint:

```
EXAMPLE / FLOW
GET /api/gifts/100?includeAllocations=true
```

Bind:

```
EXAMPLE / FLOW
giftId
    →
route

includeAllocations
    →
query
```

using explicit attributes.

Exercise 5 — Pagination

Implement:

```
EXAMPLE / FLOW
GET /api/gifts?page=1&pageSize=50
```

with:

```
EXAMPLE / FLOW
maximum page size = 200
```

and a `PagedResult<T>` response.

Exercise 6 — Filtering

Add:

```
EXAMPLE / FLOW
status

campaignId

constituentId
```

filters.

Build the EF query only from supplied filters.

Exercise 7 — Sorting

Allow:

```
giftDate

amount
```

only.

Reject/ignore unsupported sort fields rather than dynamically executing arbitrary property/SQL expressions.

Exercise 8 — ProblemDetails

Create an API integration test showing:

```
Gift not found
    →
404 ProblemDetails
```

with:

```
title

status

traceId
```

Exercise 9 — Idempotency

Design a table or persistence mechanism for:

```
IdempotencyKey

TenantId

RequestHash

CreatedGiftId

ResponseStatus
```

and ensure duplicate `POST /api/gifts` requests don't create duplicate Gifts.

Exercise 10 — Concurrency

Expose a version token from Gift.

Send:

```
If-Match: "version-x"
```

on update.

Return:

EXAMPLE / FLOW

```
412
```

when the version is stale.

Exercise 11 — Minimal API

Rewrite:

EXAMPLE / FLOW

```
GET /api/gifts/{giftId}
```

using:

C#

```
TypedResults
```

and:

C#

```
Results<Ok<GiftResponse>, NotFound>
```

Exercise 12 — OpenAPI

Register:

C#

```
builder.Services
    .AddOpenApi();
```

and in Development:

C#

```
app.MapOpenApi();
```

Inspect the generated endpoint contract and verify the Gift routes and schemas are accurately described.

PART 28 — EXACTLY 25 INTERVIEW QUESTIONS

1. What is a REST API?

A REST API is an HTTP API designed around REST architectural principles such as stateless communication, resources, standardized representations, cacheability, and a uniform interface.

REST is not a framework feature.

For example:

EXAMPLE / FLOW

```
Gift
```

may be represented by:

EXAMPLE / FLOW

```
/api/gifts/125
```

and operated on using HTTP semantics such as:

EXAMPLE / FLOW

```
GET
POST
PUT
DELETE
```

The important idea is that the API uses HTTP itself as part of the application contract instead of hiding every operation behind arbitrary RPC-style methods.

2. What is a resource in REST?

A resource is an identifiable conceptual entity exposed by the API.

In the Fundraising Platform:

EXAMPLE / FLOW

```
Gift

Campaign

Fund

Constituent
```

are natural resources.

URIs identify these resources:

EXAMPLE / FLOW

```
/api/gifts/100

/api/campaigns/20
```

The response is a representation of that resource, usually JSON.

3. What is the difference between GET and POST?

GET retrieves a representation and should use safe/read semantics.

POST submits data for processing and is commonly used to create a resource or initiate an operation.

Example:

EXAMPLE / FLOW

```
GET /api/gifts/100
    →
retrieve Gift

POST /api/gifts
    →
create Gift
```

HTTP defines safe methods as essentially read-only from the requested client semantics.

4. What does idempotent mean in HTTP?

An operation is idempotent when repeating the same request has the same intended final effect as performing it once.

HTTP defines:

EXAMPLE / FLOW

```
PUT

DELETE

safe methods such as GET
```

as idempotent.

POST is not inherently idempotent, although an application can add an idempotency mechanism.

5. What is the difference between PUT and PATCH?

PUT generally expresses the desired state of a resource at a known target URI and is idempotent.

PATCH represents a partial modification.

For example:

EXAMPLE / FLOW

```
PUT
    →
send complete Campaign update representation

PATCH
    →
change only Campaign description
```

Regardless of method, domain rules must still be enforced.

6. Why should GET not change server state?

GET is defined using safe/read semantics.

Clients, proxies, browsers, crawlers, and prefetchers may issue GET requests assuming they don't request destructive state changes.

Therefore:

EXAMPLE / FLOW

```
GET /api/gifts/100/delete
```

is poor design.

Use a state-changing method such as DELETE or POST depending on the operation.

7. When should an API return 201 Created?

Return 201 when processing successfully created a resource.

The response should normally identify the newly created resource, commonly using the `Location` header.

ASP.NET Core supports this naturally with:

C#

```
CreatedAtAction(...)
```

and HTTP defines 201 specifically for successful resource creation.

8. What is the difference between 200 and 204?

`200 OK` normally indicates successful processing and can include a response body.

`204 No Content` means the operation succeeded but no response content is being sent.

For example:

EXAMPLE / FLOW

```
GET Gift
    →
200 + Gift JSON

Complete Gift
    →
204 if no result body is needed
```

HTTP defines 204 as successful completion without response content.

9. What is the difference between 400, 404, and 409?

`400 Bad Request`:

EXAMPLE / FLOW

```
request input is invalid
```

`404 Not Found`:

EXAMPLE / FLOW

```
requested resource isn't found
```

`409 Conflict`:

EXAMPLE / FLOW

```
request conflicts with current resource state
```

For example:

EXAMPLE / FLOW

```
negative amount
    →
400

Gift ID missing
    →
404

Gift already completed
    →
possibly 409
```

RFC 9110 defines 409 as a conflict with the current state of the target resource.

10. What is [ApiController]?

`[ApiController]` enables opinionated Web API behavior for controller classes.

Important features include:

EXAMPLE / FLOW

```
Attribute routing requirement

Automatic HTTP 400 responses

Binding-source inference

Multipart/form-data inference
```

```
Problem-details behavior
```

For invalid model state, the framework automatically returns a 400 response so controller actions normally don't need to check `ModelState.IsValid`.

11. What is model binding?

Model binding converts HTTP request values into .NET action parameters and objects.

Sources include:

EXAMPLE / FLOW

```
Route

Query string

Request body

Headers

Form data

Services
```

For example:

EXAMPLE / FLOW

```
/api/gifts/100?page=2
```

can bind:

C#

```
long giftId

int page
```

automatically.

12. What is the difference between model binding and model validation?

Model binding answers:

EXAMPLE / FLOW

```
Can request data be obtained and converted into the required .NET type?
```

Model validation answers:

EXAMPLE / FLOW

```
Does the bound value satisfy validation rules?
```

For example:

EXAMPLE / FLOW

```
"abc" into decimal
    →
binding failure

-100 into decimal
```

```
    →
binding succeeds, validation may fail
```

Microsoft explicitly describes these as two separate model-state error sources.

13. What are [FromRoute], [FromQuery], and [FromBody]?

They explicitly identify where an action parameter should be bound from.

EXAMPLE / FLOW

```
[FromRoute]
    →
route values

[FromQuery]
    →
query string

[FromBody]
    →
request body
```

Other standard sources include:

EXAMPLE / FLOW

```
[FromHeader]

[FromForm]

[FromServices]
```

14. Why should API DTOs usually be separate from domain/EF entities?

API DTOs define an external contract.

Domain/EF entities contain internal concerns such as:

EXAMPLE / FLOW

```
navigation relationships

business state

internal properties

persistence mappings

methods
```

Returning entities directly can expose sensitive/internal data and tightly couple clients to persistence/domain implementation.

A DTO lets the API explicitly control what enters and leaves the system.

15. What is ActionResult<T>?

`ActionResult<T>` allows a controller action to return either:

EXAMPLE / FLOW

```
T
```

or:

EXAMPLE / FLOW

an ActionResult such as NotFound()

WEB API / REST

Example:

```
C#
public ActionResult<GiftResponse> Get(...)
```

may return:

EXAMPLE / FLOW

GiftResponse

or

404

It also lets OpenAPI-related metadata infer the successful response type from `T`.

16. When should you use `IActionResult`?

`IActionResult` is appropriate when an action can return multiple HTTP results and there isn't one strong primary response type that benefits from `ActionResult<T>`.

Example:

```
C#
public async Task<IActionResult> Delete(...)
```

may return:

EXAMPLE / FLOW

204

404

409

Microsoft documents `IActionResult` specifically for actions with multiple result possibilities.

17. What is the difference between `Results` and `TypedResults` in Minimal APIs?

`Results` helpers return:

```
C#
IResult
```

while `TypedResults` helpers return specific strongly typed HTTP-result implementations.

TypedResults improves:

EXAMPLE / FLOW

compile-time clarity

unit testing

OpenAPI metadata generation

ASP.NET Core also provides `Results<T1,T2,...>` unions for endpoints with multiple strongly typed result possibilities.

18. What is content negotiation?

Content negotiation is the process of selecting an appropriate response format based on client preferences such as the:

EXAMPLE / FLOW

```
Accept
```

header.

ASP.NET Core MVC uses output formatters to generate controller response representations.

JSON is the normal default API representation, and applications can configure 406 behavior when requested formats can't be produced.

19. What is ProblemDetails?

Problem Details is a standardized structured representation for HTTP API errors.

It typically includes:

EXAMPLE / FLOW

```
type

title

status

detail

instance
```

plus optional extensions such as:

EXAMPLE / FLOW

```
traceId
```

ASP.NET Core provides built-in Problem Details support, and `[ApiController]` uses `ValidationProblemDetails` for automatic invalid-model responses.

20. How should pagination be designed?

Large collections should not normally be returned without limits.

A simple API might use:

EXAMPLE / FLOW

```
GET /api/gifts?page=2&pageSize=50
```

and return:

EXAMPLE / FLOW

```
Items

Page

PageSize
```

```
TotalCount
```


Maximum page size should be validated.

For very large/high-change data sets, keyset/cursor pagination may scale and behave better than deep offset paging.

21. What is API idempotency and why is it important for financial APIs?

Idempotency prevents repeated identical client operations from producing unintended duplicate business effects.

Example:

EXAMPLE / FLOW

```
POST Gift
    ↓
server commits
    ↓
response lost
    ↓
client retries
```

Without protection, two Gifts could be created.

Financial APIs often combine:

EXAMPLE / FLOW

```
idempotency key

unique external transaction ID

database constraint

deduplication logic
```

to safely handle retries.

22. How can ETags help with concurrent updates?

The server returns a representation validator:

EXAMPLE / FLOW

```
ETag: "version-7"
```

Client later updates with:

EXAMPLE / FLOW

```
If-Match: "version-7"
```

If the resource has already changed to version 8, the precondition fails.

The server can respond:

EXAMPLE / FLOW

```
412 Precondition Failed
```

HTTP specifies this behavior for failed `If-Match` preconditions.

This prevents accidental lost updates.

23. What is OpenAPI in ASP.NET Core?

OpenAPI describes an API's endpoints, parameters, request/response schemas, status codes, and other contract metadata.

Current ASP.NET Core provides first-party support through:

```C#
builder.Services.AddOpenApi();

app.MapOpenApi();
```

`AddOpenApi` registers generation services, while `MapOpenApi` exposes the generated document endpoint.

24. How should API versioning be approached?

Version only when contract changes are genuinely breaking.

Common strategies include:

```EXAMPLE / FLOW
/api/v1/gifts

query-string version

header version

media-type version
```

Prefer backward-compatible evolution where possible.

For example:

```EXAMPLE / FLOW
adding an optional response property
```

may not require a new version.

But:

```EXAMPLE / FLOW
renaming/removing required fields

changing field types

changing endpoint meaning
```

may require a new contract version.

25. How would you design the Fundraising Platform Web API?

A practical API could use:

```EXAMPLE / FLOW
/api/constituents

/api/gifts

/api/campaigns

/api/funds
```

Gift routes:

EXAMPLE / FLOW

```
GET  /api/gifts
GET  /api/gifts/{giftId}
POST /api/gifts
POST /api/gifts/{giftId}/complete
POST /api/gifts/{giftId}/cancel
GET  /api/gifts/{giftId}/allocations
```

Controllers or Minimal APIs should handle:

EXAMPLE / FLOW

```
HTTP binding

HTTP validation

Status codes

DTO mapping

Protocol concerns
```

while:

EXAMPLE / FLOW

```
Application Layer
    →
orchestrates use cases

Domain
    →
protects Gift/Campaign/Fund rules

Infrastructure
    →
EF Core, SQL Server, external systems
```

Creation should return:

EXAMPLE / FLOW

```
201 + Location
```

successful commands with no output can return:

EXAMPLE / FLOW

```
204
```

missing resources:

EXAMPLE / FLOW

```
404
```

validation failures:

EXAMPLE / FLOW

```
400
```

business/concurrency conflicts:

EXAMPLE / FLOW

```
409 or 412 depending on semantics
```

WEB API / REST

Large Gift collections should use:

EXAMPLE / FLOW

```
pagination

filtering

sorting

projection
```

Critical financial POST operations should consider:

EXAMPLE / FLOW

```
idempotency

external reference uniqueness

transaction safety
```

The API should expose deliberate DTO contracts rather than EF entities, produce consistent Problem Details, propagate cancellation, document itself accurately through OpenAPI, and treat authentication, tenant isolation, and authorization as server-side trust boundaries.

PART 29 — QUICK REVISION

Concept	Meaning
Web API	HTTP interface to application capabilities
REST	Resource-oriented architectural style
Resource	Entity/concept identified by URI
Representation	JSON/XML/etc. form of resource
GET	Retrieve
POST	Submit/create/execute
PUT	Replace/update target resource
PATCH	Partial modification
DELETE	Remove target
Safe	Client doesn't request state change
Idempotent	Repeating request gives same intended effect
200	Successful response
201	Resource created
202	Accepted for later processing
204	Success with no body
400	Invalid request

Concept	Meaning
401	Authentication required/failed
403	Authenticated but forbidden
404	Resource not found
409	Resource-state conflict
412	HTTP precondition failed
415	Unsupported media type
`[ApiController]`	Opinionated controller API behavior
Model binding	HTTP → .NET parameter/object
Validation	Verify bound input rules
DTO	API boundary contract
`ActionResult<T>`	Typed controller response + alternate results
`IActionResult`	General controller result
`TypedResults`	Strongly typed Minimal API results
Content negotiation	Select response representation
ProblemDetails	Standard error representation
Pagination	Bounded collection retrieval
Idempotency key	Duplicate-request protection
ETag	HTTP representation/version validator
OpenAPI	Machine-readable API contract

PART 30 — FINAL END-TO-END FUNDRAISING API FLOW

EXAMPLE / FLOW

```
CLIENT
   ↓
POST /api/gifts
   ↓
HTTP Request
   ↓
Routing
   ↓
Authentication / Authorization
   ↓
GiftsController.Create
   ↓
Model Binding
   ↓
CreateGiftRequest
   ↓
Boundary Validation
   ↓
CreateGiftService
   ↓
Gift.Create()
   ↓
```

```
Gift business invariants
   ↓
Repository
   ↓
EF Core
   ↓
SQL Server
   ↓
Gift created
   ↓
GiftResponse DTO
   ↓
CreatedAtAction
   ↓
201 Created
   ↓
Location: /api/gifts/{giftId}
   ↓
CLIENT
```

Read flow:

EXAMPLE / FLOW

```
GET /api/gifts/125
   ↓
Route GiftId
   ↓
GiftQueries
   ↓
AsNoTracking
   ↓
Projection
   ↓
GiftResponse
   ↓
200 OK
```

Missing resource:

EXAMPLE / FLOW

```
GET /api/gifts/999999
   ↓
No result
   ↓
404 ProblemDetails
```

Validation failure:

EXAMPLE / FLOW

```
POST /api/gifts

Amount = -100
   ↓
Model Binding
   ↓
Validation
   ↓
[ApiController]
   ↓
400 ValidationProblemDetails
   ↓
Controller action not executed
```

Concurrency flow:

EXAMPLE / FLOW

```
GET Gift
   ↓
ETag = version-7
   ↓
Another user updates Gift
   ↓
Client sends PUT with
If-Match: version-7
   ↓
Current version = version-8
   ↓
412 Precondition Failed
```

Financial retry flow:

EXAMPLE / FLOW

```
POST Gift
+
Idempotency-Key
   ↓
Server processes request
   ↓
Response lost
   ↓
Client retries same key
   ↓
Server recognizes prior operation
   ↓
No duplicate Gift
```

The final mental model is:

EXAMPLE / FLOW

```
URI
    =
Which resource?

HTTP method
    =
What operation semantics?

Request DTO
    =
What input contract?

Status code
    =
What happened?

Response DTO
    =
What representation is returned?

ProblemDetails
    =
How failure is represented?

Application/Domain
```

```
    =
What business operation actually occurs?
```


The central lesson is:

> **A good ASP.NET Core Web API does more than expose controller methods over HTTP. It treats HTTP itself as part of the contract: resources have stable URIs, methods preserve their intended semantics, status codes communicate outcomes, DTOs protect boundaries, validation rejects malformed input, domain rules remain inside the application/domain model, large collections are paginated and projected efficiently, retries and concurrency are handled deliberately, and OpenAPI accurately documents the contract clients depend on.**

Next roadmap topic: Topic 40 — ASP.NET Core Security, Integration and Production Backend Design.